

COMPTE-RENDU DE TRAVAUX PRATIQUES

Prédiction du rendement agricole par machine learning accéléré sur GPU

Auteur: Anass Bouras
Jaghdor Ilyas
Hamza Belkadi

1. Introduction et objectif du TP

L'agriculture mondiale fait face aujourd'hui à des défis majeurs liés au changement climatique, à la croissance démographique et à la nécessité de rationaliser l'utilisation des ressources naturelles et chimiques (eau, pesticides). Dans ce contexte, la capacité à prédire avec précision le rendement des cultures (exprimé en hectogrammes par hectare, hg/ha) revêt une importance stratégique majeure pour la sécurité alimentaire mondiale et la planification économique locale.

Ce compte-rendu décrit les travaux pratiques visant à concevoir, évaluer et interpréter un pipeline de Machine Learning complet pour la prédiction du rendement agricole. Les objectifs pédagogiques de ces travaux pratiques sont les suivants :

- Prendre en main la suite RAPIDS de NVIDIA, en particulier cuDF pour la manipulation rapide de données sous forme de DataFrames et cuML pour les algorithmes de Machine Learning accélérés par le processeur graphique (GPU).
- Mettre en œuvre XGBoost (Extreme Gradient Boosting) en configurant l'accélération matérielle GPU (via la méthode des histogrammes 'hist' et le périphérique 'cuda').
- Comparer les performances de modèles non-linéaires avancés (Random Forest, XGBoost) à celles d'un modèle de référence linéaire classique (Régression Linéaire) à l'aide de métriques standards telles que la racine de l'erreur quadratique moyenne (RMSE) et le coefficient de détermination (R^2).
- Lever l'opacité (l'effet « boîte noire ») des modèles complexes d'ensemble en utilisant la bibliothèque SHAP (Shapley Additive exPlanations) pour expliquer les prédictions tant au niveau global qu'individuel.

2. Description du projet et du jeu de données

Le projet exploite des données issues de l'Organisation des Nations unies pour l'alimentation et l'agriculture (FAO FAOSTAT), disponibles sous la forme d'un dataset sur Kaggle (crop-yield-prediction-dataset). Ce dataset rassemble des données géographiques, temporelles, climatiques et d'intrants agricoles sur une période s'étendant de 1990 à 2013, à travers 101 pays et 10 types de cultures.

2.1 Variables explicatives (Features) et variable cible (Target)

Les données comportent les variables suivantes :

- country (catégorielle, 101 modalités) : Le pays où les mesures ont été effectuées.
- crop (catégorielle, 10 modalités) : Le type de culture concerné (ex. Maize, Potatoes, Rice paddy, Sorghum, Soybeans, etc.).
- year (temporelle) : L'année de récolte.
- average_rain_fall_mm_per_year (numérique) : Le niveau moyen annuel de précipitations en millimètres par an.
- pesticides_tonnes (numérique) : La quantité de pesticides utilisée en tonnes sur le territoire concerné.
- avg_temp (numérique) : La température moyenne annuelle en degrés Celsius (°C).
- yield_hg_ha (numérique, Target) : Rendement agricole mesuré en hectogrammes par hectare (hg/ha). Il s'agit de la variable continue que les modèles cherchent à prédire.

2.2 Statistiques descriptives et distribution de la variable cible

Après nettoyage des doublons et suppression des lignes comportant des valeurs manquantes ou des rendements aberrants (inférieurs ou égaux à 0), le dataset final comporte 28 242 lignes. Les statistiques descriptives de la variable yield_hg_ha sont résumées ci-dessous :

Métrique statistique	Valeur (hg/ha)
Nombre d'observations (count)	28 242
Moyenne (mean)	77 053
Écart-type (std)	84 957
Minimum (min)	50
25ème centile (Q1)	19 919
Médiane (50%)	38 295
75ème centile (Q3)	104 677
Maximum (max)	501 412

Commentaire sur la distribution : La moyenne (77 053 hg/ha) est largement supérieure à la médiane (38 295 hg/ha), ce qui indique une distribution fortement asymétrique vers la droite (skewed right). Cette

disparité montre que la majorité des récoltes enregistre des rendements modérés (Q1 à Q3 s'étendant de ~20k à ~105k hg/ha), tandis qu'une minorité bénéficie de rendements exceptionnels pouvant atteindre plus de 500 000 hg/ha, tirant ainsi la moyenne vers le haut. Cette asymétrie justifie l'utilisation de modèles robustes basés sur les arbres de décision, moins sensibles aux valeurs extrêmes et aux asymétries de distribution.

3. Étapes du projet (détail méthodologique)

3.1 Vérification du GPU

Avant de démarrer le code, il est nécessaire de s'assurer de la présence d'un GPU compatible pour exécuter la suite RAPIDS cuML et accélérer XGBoost. La commande suivante est exécutée :

```
!nvidia-smi --query-gpu=name,memory.total --format=csv,noheader
```

Résultat obtenu dans le notebook : Tesla T4, 15360 MiB. Le GPU Tesla T4 alloué offre environ 15 Go de mémoire vidéo (VRAM), ce qui est largement suffisant pour notre dataset.

3.2 Installation de RAPIDS et des dépendances

L'installation s'effectue via un script automatisé qui détecte la version de CUDA et installe la version correspondante de RAPIDS (temps d'exécution : environ 10 minutes) :

```
import subprocess, sys
try:
    import cuml
    print(f'cuml {cuml.__version__} déjà installé, on passe.')
except ImportError:
    subprocess.run(['git', 'clone', '--quiet', 'https://github.com/rapidsai/rapidsai-csp-utils.git'], check=True)
    subprocess.run([sys.executable, 'rapidsai-csp-utils/colab/pip-install.py'], check=True)

# Installation complémentaire des autres packages
!pip install -q kaggle xgboost>=2.0 shap>=0.45
```

3.3 Imports et bascule automatique GPU/CPU

Pour assurer la portabilité du code entre les machines équipées d'un GPU NVIDIA et celles fonctionnant uniquement sur processeur (CPU), le notebook met en place une structure try/except. Si RAPIDS est installé, les algorithmes de cuML (GPU) sont chargés. Dans le cas contraire, le script bascule sur scikit-learn (CPU) :

```
USE_GPU = False
try:
    import cudf
    from cuml.ensemble import RandomForestRegressor as RF
    from cuml.linear_model import LinearRegression as LR
    USE_GPU = True
```

```

print(f'✓ RAPIDS cuML {__import__("cuml").__version__} — mode GPU')
except ImportError:
    from sklearn.ensemble import RandomForestRegressor as RF
    from sklearn.linear_model import LinearRegression as LR
    print('⚠ RAPIDS absent — fallback scikit-learn CPU')

print(f'XGBoost {xgb.__version__} | SHAP {shap.__version__}')

```

Sortie affichée dans le notebook :

- ✓ RAPIDS cuML 26.02.000 — mode GPU
- XGBoost 3.2.0 | SHAP 0.51.0

3.4 Chargement du dataset

Le dataset est directement importé depuis Kaggle à l'aide de l'API officielle :

- Téléchargement du fichier kaggle.json contenant le token d'authentification API de l'utilisateur.
- Création du répertoire de configuration sécurisé /root/.kaggle/ et écriture du token.
- Téléchargement et décompression automatique du jeu de données patelris/crop-yield-prediction-dataset.

Ensuite, les étapes de nettoyage suivantes sont appliquées en Pandas :

- Renommage harmonieux des colonnes : Area -> country, Item -> crop, Year -> year, hg/ha_yield -> yield_hg_ha.
- Suppression des doublons via df.drop_duplicates().
- Suppression des lignes présentant des valeurs manquantes dans les colonnes critiques ('yield_hg_ha', 'average_rain_fall_mm_per_year', 'avg_temp').
- Filtrage des rendements pour ne conserver que les valeurs strictement positives.

3.5 Analyse exploratoire des données (EDA)

L'EDA consiste à tracer des histogrammes de répartition des variables pour comprendre les plages de valeurs et les profils de distribution. Le notebook produit une figure composite contenant quatre graphiques :

[FIGURE : Visualisations de l'EDA - Distribution des Variables]

1. Distribution du rendement (vert) : Distribution asymétrique à droite avec ligne rouge représentant la médiane à 38 295 hg/ha. Majorité de faibles rendements et longue traîne.
2. Distribution des précipitations (bleu) : Présence de plusieurs pics (distribution multimodale), reflétant les climats très contrastés des pays du dataset (régions sèches vs humides).
3. Distribution de la température moyenne (rouge) : Distribution bimodale. Un pic important se situe autour de 25°C (pays tropicaux/équatoriaux) et un autre pic plus étalé autour de 10-15°C (pays tempérés).
4. Distribution des pesticides en échelle logarithmique (orange) : Représentation de $\text{Log}(1 + \text{pesticides_tonnes})$ montrant une distribution asymétrique, avec une grande proportion de zones agricoles utilisant des quantités de pesticides faibles à moyennes, et quelques zones à très forte consommation.

3.6 Prétraitement et Feature Engineering

Les variables catégorielles (pays et cultures) sont encodées en entiers à l'aide de LabelEncoder. Ensuite, deux nouvelles variables synthétiques sont créées pour capturer des relations temporelles et dynamiques :

- `year_trend = year - year.min()` : Normalisation temporelle mesurant le nombre d'années écoulées depuis le début des enregistrements (1990). Cela permet aux modèles de modéliser une tendance linéaire ou non-linéaire temporelle globale.
- `crop_year = crop * year_trend` : Variable d'interaction qui associe l'identifiant numérique de la culture à la tendance temporelle, permettant de capturer des évolutions de rendement spécifiques à chaque culture au fil des ans.

Le jeu de caractéristiques final FEATURES comprend ainsi 8 colonnes explicatives :

```
FEATURES = [  
    'country', 'crop', 'year', 'year_trend', 'crop_year',  
    'average_rain_fall_mm_per_year', 'pesticides_tonnes', 'avg_temp'  
]  
TARGET = 'yield_hg_ha'  
  
# Dimensions des matrices : X : (28242, 8) | y : (28242,)
```

3.7 Split Train/Test et Normalisation

Le dataset est découpé en 80 % pour l'entraînement (22 593 lignes) et 20 % pour le test (5 649 lignes) avec un `random_state=42`. Une étape clé concerne la normalisation (`StandardScaler`) :

- La normalisation par `StandardScaler` est calculée sur le train et appliquée au train et au test.
- Remarque critique : Cette normalisation est utilisée UNIQUEMENT pour le modèle de Régression Linéaire. Les modèles Random Forest et XGBoost n'exigent pas de normalisation car les séparations dans les arbres de décision dépendent uniquement de l'ordre des valeurs (invariance d'échelle).

3.8 Transfert en mémoire GPU

Pour les modèles gérés par RAPIDS, les matrices NumPy et DataFrames Pandas sont copiés en mémoire GPU (VRAM) sous forme d'objets cuDF (cuDF DataFrames et cuDF Series). Cela évite les goulets d'étranglement de transfert de données CPU-GPU pendant l'entraînement :

```
if USE_GPU:  
    Xtr = cudf.DataFrame(X_train, columns=FEATURES)  
    Xte = cudf.DataFrame(X_test, columns=FEATURES)  
    ytr = cudf.Series(y_train)  
    yte = cudf.Series(y_test)  
    Xtr_s = cudf.DataFrame(X_train_sc, columns=FEATURES)  
    Xte_s = cudf.DataFrame(X_test_sc, columns=FEATURES)
```

3.9 Entraînement des modèles

A) Random Forest (GPU cuML)

- Hyperparamètres : $n_estimators=100$ (100 arbres), $max_depth=16$ (profondeur maximale des arbres), $random_state=42$. Les paramètres spécifiques au GPU sont également configurés : $n_bins=32$ (nombre de bins pour discrétiser les variables continues lors de la recherche des splits) et $n_streams=4$ (nombre de flux CUDA pour le parallélisme des calculs).
- Temps d'exécution : 1.1s (grâce à l'accélération matérielle massive de cuML).
- Performances en test : $RMSE = 10\,127\text{ hg/ha}$, $R^2 = 0.9859$.

B) XGBoost (GPU)

- Hyperparamètres : $n_estimators=300$, $max_depth=8$, $learning_rate=0.05$, $subsample=0.8$ (proportion de lignes échantillonnées pour chaque arbre), $colsample_bytree=0.8$ (proportion de colonnes échantillonnées), $tree_method='hist'$ (méthode de construction d'arbres basée sur les histogrammes pour GPU), $device='cuda'$ (sélection explicite du GPU), $random_state=42$.
- Temps d'exécution : 1.1s.
- Performances en test : $RMSE = 11\,081\text{ hg/ha}$, $R^2 = 0.9831$.

C) Régression Linéaire (GPU cuML)

- Particularité : Entraînée sur les données normalisées (StandardScaler).
- Temps d'exécution : 0.4s.
- Performances en test : $RMSE = 82\,452\text{ hg/ha}$, $R^2 = 0.0628$.

4. Résultats et comparaison des modèles

Les performances de chaque modèle ont été évaluées sur le jeu d'entraînement et sur le jeu de test indépendant afin d'estimer leur capacité de généralisation. Les résultats sont synthétisés dans le tableau ci-dessous :

Modèle	RMSE Train	R^2 Train	RMSE Test	R^2 Test	Temps (s)
Random Forest ★	4 109	0.9977	10 127	0.9859	1.1s
XGBoost	6 509	0.9941	11 081	0.9831	1.1s
Régression Linéaire	82 297	0.0604	82 452	0.0628	0.4s

4.1 Analyse et commentaires

- Identification du meilleur modèle : Le modèle Random Forest se classe premier (marqué par une étoile ★), obtenant le RMSE le plus bas (10 127 hg/ha) et le R^2 le plus élevé (0.9859) sur le jeu de test. XGBoost se situe juste derrière avec des performances très proches (RMSE de 11 081 hg/ha et R^2 de 0.9831). Les deux modèles basés sur les arbres surclassent radicalement la Régression Linéaire.
- Échec de la Régression Linéaire : La Régression Linéaire affiche un coefficient de détermination R^2 extrêmement faible (0.0628 sur le jeu de test). Cela indique que le rendement agricole est régi par des relations hautement complexes et non-linéaires ainsi que par des interactions croisées (comme le climat en fonction de la culture) que l'équation linéaire ne peut pas capturer.
- Surapprentissage (Overfitting) : Nous observons que le Random Forest présente un RMSE Train (4 109 hg/ha) nettement inférieur à son RMSE Test (10 127 hg/ha). Ce phénomène est classique pour les modèles d'arbres très profonds (max_depth=16) qui s'ajustent très finement au bruit des données d'entraînement. Néanmoins, sa capacité de généralisation reste la meilleure parmi les trois modèles.
- Avantage de l'accélération GPU : L'entraînement d'une forêt aléatoire de 100 arbres à une profondeur de 16 sur 22 593 lignes ne prend que 1.1s sous GPU grâce à RAPIDS cuML. En mode CPU classique (scikit-learn), une telle tâche prendrait plusieurs dizaines de secondes, voire minutes, ce qui illustre le gain de productivité pour les phases de tuning d'hyperparamètres.

[FIGURE : Comparaison graphique des performances des modèles]

1. Graphique en barres RMSE (gauche) : Met en évidence le gouffre de performance entre la Régression Linéaire (erreur proche de 82k hg/ha) et les modèles non-linéaires d'arbres (erreurs proches de 10k-11k hg/ha).
2. Graphique en barres R^2 (centre) : Affiche des barres proches de 1.0 (0.986 et 0.983) pour Random Forest et XGBoost, tandis que la barre de la Régression Linéaire stagne au niveau du sol (0.063).
3. Nuage de points Réel vs Prédit pour Random Forest (droite) : Représente 800 observations du jeu de test. Les points bleus s'alignent étroitement le long de la ligne diagonale rouge pointillée ('Parfait'), indiquant une excellente précision globale. On remarque une dispersion légèrement accrue pour les rendements très élevés (au-dessus de 300 000 hg/ha), où le modèle a tendance à légèrement sous-estimer les valeurs extrêmes.

5. Interprétation avec SHAP (sur XGBoost)

5.1 Principe et intérêt de SHAP

Les modèles d'ensemble comme XGBoost ou Random Forest sont généralement qualifiés de modèles en « boîte noire » car il est impossible de comprendre intuitivement comment les décisions sont prises à travers des centaines d'arbres complexes. SHAP (Shapley Additive exPlanations) résout ce problème en s'appuyant sur la théorie des jeux coopératifs. Il attribue à chaque variable explicative une valeur de contribution (valeur de Shapley) pour une prédiction donnée. L'intérêt de SHAP est double :

- Propriété d'additivité locale : Pour une observation donnée, la somme des valeurs SHAP de toutes les features ajoutée à la valeur moyenne de prédiction globale (valeur de base) est exactement égale à la prédiction finale du modèle.

- Interprétation globale et locale : SHAP permet d'obtenir une vision macro-économique des features les plus importantes pour l'ensemble du dataset (importance globale) tout en expliquant pas à pas chaque prédiction individuelle (importance locale).

5.2 Importance globale des caractéristiques (Global Feature Importance)

L'analyse globale est réalisée en calculant la moyenne de la valeur absolue des valeurs SHAP pour chaque variable sur un échantillon de 800 observations de test :

[FIGURE : Importance globale des Features (SHAP Summary Bar Plot)]

Le graphique affiche des barres horizontales bleues classées par ordre décroissant d'impact moyen sur le modèle :

1. *crop (culture)* : Impact majeur, atteignant environ 50 000 hg/ha en moyenne. C'est le facteur le plus discriminant.
2. *pesticides_tonnes* : Impact moyen d'environ 10 000 hg/ha.
3. *avg_temp (température moyenne)* : Impact moyen d'environ 9 000 hg/ha.
4. *crop_year (interaction)* : Impact moyen d'environ 8 000 hg/ha.
5. *year (année)* : Impact moyen d'environ 7 000 hg/ha.
6. *average_rain_fall_mm_per_year* : Impact moyen d'environ 6 500 hg/ha.
7. *country (pays)* : Impact moyen d'environ 6 000 hg/ha.
8. *year_trend (tendance de l'année)* : Impact très faible, environ 1 500 hg/ha.

Analyse : La nature de la culture (*crop*) domine largement les prédictions. Cela est cohérent sur le plan agronomique : le rendement intrinsèque d'une pomme de terre (poids élevé par hectare) est naturellement très différent de celui du soja ou du sorgho, quelles que soient les conditions climatiques.

5.3 Décomposition d'une prédiction individuelle (Waterfall Plot)

Pour comprendre le comportement local du modèle XGBoost, le notebook extrait une observation spécifique dont la prédiction est la plus proche possible de la médiane des prédictions. Pour cette observation :

- Rendement réel constaté : 28 621 hg/ha
- Rendement prédit par le modèle XGBoost : 37 055 hg/ha
- Valeur attendue moyenne (Base Value $E[f(X)]$) : 77 069.07 hg/ha

Le graphique de type « cascade » (waterfall plot) explique visuellement comment le modèle passe de 77 069.07 hg/ha à 37 055 hg/ha en additionnant les contributions positives (en rouge) et négatives (en bleu) de chaque variable :

[FIGURE : Waterfall Plot - Décomposition d'une prédiction individuelle]

Base Value $E[f(X)] = 77\,069.07$ hg/ha

Contributions successives :

- *crop* = 4 (culture encodée à 4) : -29 239.24 hg/ha (effet réducteur majeur)
- *crop_year* = 84 (interaction) : -8 869.57 hg/ha (effet réducteur)
- *avg_temp* = 19.85 °C : -7 489.68 hg/ha (effet réducteur)
- *country* = 36 (pays encodé à 36) : -4 043.76 hg/ha (effet réducteur)
- *average_rain_fall_mm_per_year* = 1996 mm : -1 528.60 hg/ha (effet réducteur)
- *year* = 2011 (récolte en 2011) : +9 045.29 hg/ha (effet amplificateur important)

• *year_trend = 21 (21 ans d'historique) : +1 939.70 hg/ha (effet amplificateur)*
 • *pesticides_tonnes = 16540.711 : +172.14 hg/ha (effet amplificateur très marginal)*
 Prédiction finale : 37 055 hg/ha (Somme = 77 069.07 - 29 239.24 - 8 869.57 - 7 489.68 - 4 043.76 - 1 528.60 + 9 045.29 + 1 939.70 + 172.14 = 37 055.35)

Interprétation agronomique locale : Pour cette parcelle cultivée en 2011 (year = 2011), le rendement prédit de 37 055 hg/ha est nettement inférieur à la moyenne mondiale (77 069). L'analyse SHAP révèle que cette contre-performance est principalement due au type de culture (crop = 4, baisse de -29k hg/ha) qui a un rendement de base faible, combiné à des températures moyennes élevées (19.85 °C, baisse de -7.5k hg/ha) qui ont pu stresser les cultures. Cependant, le fait que la récolte ait eu lieu en 2011 (effet positif de +9k hg/ha) suggère que l'amélioration des pratiques agricoles au fil du temps (machinerie, semences) a permis d'atténuer ces facteurs négatifs.

6. Conclusion

Ces travaux pratiques ont mis en évidence la supériorité des modèles de régression non-linéaires basés sur des arbres (Random Forest et XGBoost) pour modéliser le rendement agricole, un phénomène régi par des variables aux interactions complexes et asymétriques. L'intégration de la suite logicielle RAPIDS a permis d'entraîner et d'évaluer ces modèles complexes en un temps record (1.1s), prouvant l'intérêt de l'accélération GPU pour les pipelines de science des données.

Néanmoins, le projet présente certaines limites : le dataset s'arrête en 2013 et ne comporte qu'un nombre restreint de variables environnementales macroscopiques. Pour aller plus loin, plusieurs perspectives peuvent être envisagées :

- Enrichir les caractéristiques en intégrant des variables agronomiques locales : texture du sol, taux de nutriments (azote, phosphore, potassium), index d'irrigation, et index de végétation par satellite (NDVI).
- Utiliser des méthodes de validation croisée adaptées aux séries temporelles (Time Series Split) pour évaluer la capacité du modèle à prédire les rendements sur des années futures non observées pendant l'entraînement.
- Déployer le modèle sous forme d'application interactive d'aide à la décision pour les agriculteurs, couplée aux explications SHAP locales pour expliciter les prévisions.

7. Questions de compréhension

Ces questions ont pour but de valider la compréhension des concepts théoriques et pratiques mis en œuvre dans ce TP.

Question 1 (Choix techniques) : Pourquoi est-il indispensable de disposer d'un accélérateur matériel (GPU) dans le cadre de ce TP, et quelles bibliothèques logicielles permettent son exploitation ?

Question 2 (Normalisation) : Pourquoi la normalisation des variables (StandardScaler) a-t-elle été appliquée uniquement pour la Régression Linéaire et non pour Random Forest ou XGBoost ?

Question 3 (Interprétation des métriques) : Comment expliquez-vous la différence spectaculaire de performance (R^2 de 0.986 vs 0.063) entre Random Forest et la Régression Linéaire sur le jeu de test ?

Question 4 (SHAP local) : Dans le waterfall plot de l'analyse individuelle, que représente la 'Base Value' (valeur de base) et comment s'articulent les contributions des features pour donner la prédiction finale ?

Question 5 (Impact climatique) : À la lecture des résultats SHAP globaux et locaux, quel est l'impact de la température moyenne ('avg_temp') et comment s'explique-t-il agronomiquement ?

Question 6 (Reproductibilité et méthodologie) : Quel est le rôle du paramètre 'random_state=42' utilisé lors du split train/test et de la définition des modèles, et pourquoi est-il crucial en science des données ?

Question 7 (Synthèse d'ouverture) : Quel est l'intérêt de comparer et potentiellement de combiner (via du Stacking ou du Blending) plusieurs modèles de familles différentes pour résoudre ce problème de prédiction ?